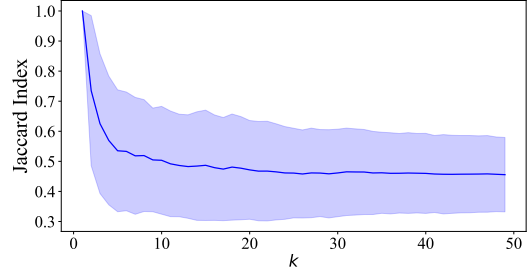


(a) T-SNE visualization of CIFAR-10 [47] with embedding models VGGNet [72] (left) and DenseNet [40] (right): hand-written '5' images are embedded further away from '9' by VGGNet while the opposite is observed in DenseNet embeddings.



(b) Consistency of top- k search over SciFact with NV-Embed V2 and OpenAI-Ada (blue line is the average of 10 random queries; shaded area shows the variance.)

Fig. 3. Are real-life vector databases (approximately) isometric?

integrated database. Moreover, the isometry f_{iso} is easy to deduce by aligning $\mathcal{D}_1[\mathcal{O}_\cap]$ with $\mathcal{D}_2[\mathcal{O}_\cap]$ via embedding alignment, a core task in robotics (rigid motion transformation) [3], computer vision (Procrustes Analysis) [69, 76] and cross-lingual word embedding alignment [5, 44, 63, 74]. We illustrate such integration function $\mathcal{T}$ by drawing parallel with image object correlation in Fig. 2 (a-b).

3 Evaluating the Isometry Hypothesis

While we have a viable solution for vector database integration by making a connection with embedding alignment via isometries, it critically depends on the assumption that Hypothesis A holds for vector databases, which we will examine closely next. Specifically, we aim to answer two questions (Q1) and (Q2) below, corresponding to part (1) and part (2) of Hypothesis A, respectively:

(Q1) Do different embedding models generate (approximately) isometric vectors of the same data?

(Q2) How well does top- k vector search perform over the integrated vector databases based on the align-to-merge approach suggested by Hypothesis A (as also illustrated in Fig. 2b)?

3.1 Are Vector Databases Isometric (Q1)?

We first examine question (Q1) to see if the same dataset (*i.e.*, $\mathcal{O}_\cap$ in Section 2) has approximately isometric vector representations embedded by different models. We use two evaluation methods: (a) we visualize vector databases of the same dataset with T-SNE [78] and examine if their visualizations are isometric; and (b) we compare the top- k similarity search results implemented by different vector databases of the same dataset. We used both image and text data.

(1) Visualizing vector databases. We first visualize two vector databases embedded from the CIFAR-10 dataset, which is commonly used for evaluating image classification methods [47].

More specifically, CIFAR-10 consists of 60,000 color images categorized into 10 distinct classes. We use all the images as the set $\mathcal{O}$ of data items, and generate two vector databases $\text{emb}_1(\mathcal{O})$ and $\text{emb}_2(\mathcal{O})$ of $\mathcal{O}$ with ResNet [37] and DenseNet [39], respectively. The T-SNE [78] visualization of $\text{emb}_1(\mathcal{O})$ and $\text{emb}_2(\mathcal{O})$ is shown in the Fig. 3a. Vectors of images from the same category are highlighted in the same color in both vector databases, and are numbered from 0 to 9. We examine the proximity between the classes in the T-SNE visualization of both vector databases such that small noises in the embedding vectors are subsumed within the classes.

Observation. If Hypothesis A holds, we would see that the classes are distributed similarly in the visualization across the two vector databases. However, this is not the case as shown by Fig. 3a. For

instance, class 8 is closer to class 9 than class 5 in VGGNet embedding vectors, while the opposite is shown in DenseNet. This shows that, VGGNet and DenseNet vectors are not isometric even if we ignore perturbation in short vector distances and zoom out to the class level to subsume distance noise.

(2) Consistency of top- k similarity search. We further examine the isometry hypothesis by looking into the consistency of top- k similarity search. The rationale is simple: if two vector databases $\text{emb}_1(O)$ and $\text{emb}_2(O)$ of the same dataset O produced by embedding models emb_1 and emb_2 , respectively, are isometric, for any query item $q \in O$, its top- k similarity search, implemented as top- k vector search of $\text{emb}_1(q)$ over $\text{emb}_1(O)$ and that of $\text{emb}_2(q)$ over $\text{emb}_2(O)$ shall be consistent and identify the same objects that are deemed as the most similar to q in O , where $\text{emb}_1(q)$ and $\text{emb}_2(q)$ are the vectors of q embedded by emb_1 and emb_2 , respectively.

To do this, we use the SciFact text dataset [80] as O , which consists of 6,000 scientific article abstracts. We use embedding models NV-Embed-V2 [50] and OpenAI-Ada [66] as emb_1 and emb_2 to create two vector databases $\text{emb}_1(O)$ and $\text{emb}_2(O)$ for SciFact, respectively. To measure the consistency of top- k search results, we use Jaccard Index [17]. Specifically, for an item $q \in O$, let $R_1 \subseteq O$ be the k items that are encoded by the top- k vector search results with query vector $\text{emb}_1(q)$ over $\text{emb}_1(O)$; similarly, let $R_2 \subseteq O$ be the items identified by $\text{emb}_2(q)$ over $\text{emb}_2(O)$. Then we define the *consistency* of top- k search with $\text{emb}_1(O)$ and $\text{emb}_2(O)$ for q as the Jaccard Index of sets R_1 and R_2 , calculated as $|R_1 \cap R_2|/|R_1 \cup R_2|$.

Intuitively, the consistency of top- k search with $\text{emb}_1(O)$ and $\text{emb}_2(O)$ ranges in $[0, 1]$ and a larger consistency ratio indicates a higher degree of alignment between the retrieval outcomes from the two different embedding representations of O .

Observation. Figure 3b shows the average consistency ratio between top- k results over NV-Embed-V2 vectors and Openai Ada vectors for 10 random query questions, with varying k . The results tell us that Hypothesis A does not hold for NV-Embed-V2 and OpenAI-Ada embedding vectors, as the consistency between search results over NV-Embed-V2 and OpenAI-Ada embedding vectors is mostly low (around 0.5), unless k is very small (e.g., less than 5).

Answer to Q1. From these experiments, we observe that vector databases of the same set of data items are not necessarily isometric if they are generated by different embedding models. Moreover, from (1) we can see that this non-isometric phenomenon cannot be simply attributed to embedding noises that perturbate the “positions” of the vectors in the space as they are not isometric even if we zoom out and view the dataset from a quite coarse resolution.

3.2 Vector Search over Integrated Databases (Q2)

We next examine evaluation question (Q2) to see how well top- k vector search performs over the integrated vector database produced by align-to-merge approach sketched in Section 2. We first present the approach in detail and then evaluate its performance for Q2.

Align-to-merge (A2M) for vector database integration. Recall notations from Section 2 that O_1 and O_2 are two sets of objects, encoded by two embedding functions emb_1 and emb_2 as vector databases $\mathcal{D}_1$ and $\mathcal{D}_2$, respectively. Let $O_\cap = O_1 \cap O_2$. Denote by $\mathcal{X}$ and $\mathcal{Y}$ the vectors in $\mathcal{D}_1$ and $\mathcal{D}_2$ that encode objects in $O_\cap$, respectively. Assume w.l.o.g. that $\mathcal{X} = \{\mathbf{x}_1, \dots, \mathbf{x}_m\}$ and $\mathcal{Y} = \{\mathbf{y}_1, \dots, \mathbf{y}_m\}$, and vectors $\mathbf{x}_i$ and $\mathbf{y}_i$ encode the same object in $O_\cap$ ($i \in [1, m]$).

The align-to-merge (A2M) approach operates in two steps. In the first step, it computes a best fit isometry function f_{iso} from $\mathcal{X}$ to $\mathcal{Y}$, as shown in Fig. 2b, to align $\mathcal{D}_1$ with $\mathcal{D}_2$ by using the fact that their subsets $\mathcal{X}$ and $\mathcal{Y}$ encode the same objects in $O_\cap$. According to Hypothesis A, when $O_\cap$ is sufficiently large, it gives us a mapping f_{iso} that determines an integration of $\mathcal{D}_1$ into $\mathcal{D}_2$ as $f_{\text{iso}}(\mathcal{D}_1) \cup (\mathcal{D}_2 \setminus \mathcal{Y})$, serving as an approximation of the vector database that emb_2 would generate for $O_\cap$.

Following Procrustes Analysis in computer vision [69, 76] and cross-lingual word embedding alignment [44, 74], f_{iso} can be formulated as $f(\mathbf{x}) = s * R(\mathbf{x} + \mathbf{t})$, where R is an orthogonal matrix of dimension $d \times d$ that represents rotation and reflection, $\mathbf{t}$ is a d -dimensional vector for translation, and s is for uniform scaling, and finding the best f_{iso} that maps $\mathcal{X}$ to $\mathcal{Y}$ reduces to an optimization problem:

$$\arg \min_{s, R, \mathbf{t}} \sum_{i=1}^m \|sR(\mathbf{x}_i + \mathbf{t}) - \mathbf{y}_i\|^2, \quad (1)$$

where $\|\cdot\|$ is the L2 norm of vectors. Intuitively, f_{iso} transforms $\mathcal{X}$ via isometry (*i.e.*, R and $\mathbf{t}$) extended with uniform scaling (*i.e.*, s) such that vectors of $\mathcal{X}$ are best aligned with their targets in $\mathcal{Y}$, with the sum of square of mis-alignment distances minimized. When $\mathcal{X}$ is exactly isometric to $\mathcal{Y}$, $f_{\text{iso}}(\mathbf{x}_i)$ is $\mathbf{y}_i$.

A nice property of this formulation is that it has a closed-form optimal solution. Let the SVD (Singular Value Decomposition) [24] of $\mathcal{Y}\mathcal{X}^T$ be $U\Sigma V^T$, where we view $\mathcal{X}$ as a $d \times m$ matrix with $\mathbf{x}_i$ ($i \in [1, m]$) as column vectors; similarly, $\mathcal{Y}$ is a $d \times m$ matrix that has $\mathbf{y}_1, \dots, \mathbf{y}_m$ as its column vectors. Then f_{iso} is determined by the optimal R , $\mathbf{t}$ and s as $f_{\text{iso}}(\mathbf{x}) = s_{\text{opt}} * R_{\text{opt}}(\mathbf{x} + \mathbf{t}_{\text{opt}})$:

- $R_{\text{opt}} = UV^T$, where $\text{SVD}(\mathcal{Y}\mathcal{X}^T) = U\Sigma V^T$,
- $s_{\text{opt}} = \text{trace}(\Sigma)/\text{trace}(\mathcal{X}^T\mathcal{X})$, and
- $\mathbf{t}_{\text{opt}} = \bar{\mathbf{y}} - \bar{\mathbf{x}}$, where $\bar{\mathbf{x}} = \sum_{i=1}^m \mathbf{x}_i/m$ and $\bar{\mathbf{y}} = \sum_{i=1}^m \mathbf{y}_i/m$.

Evaluation of Q2. We next examine evaluation question (Q2) to see how well top- k vector search performs over the integrated vector database $f_{\text{iso}}(\mathcal{D}_1) \cup (\mathcal{D}_2 \setminus \mathcal{Y})$ computed by A2M.

To do this, we use four public text retrieval benchmarks [77], each consisting of a set of text documents along with built-in query-answer pairs (q, ans) , where q is a query text and ans is the ground-truth answer documents to q validated by the benchmark.

We randomly partition each dataset into $\mathcal{O}_1$ and $\mathcal{O}_2$ with $\mathcal{O}_1 \cap \mathcal{O}_2$ containing randomly picked documents accounting for 1/3 of the dataset size; built-in benchmark ans documents are evenly distributed in $\mathcal{O}_1$ and $\mathcal{O}_2$. We then use two embedding models emb_1 and emb_2 to encode $\mathcal{O}_1$ and $\mathcal{O}_2$, respectively, as $\mathcal{D}_1$ and $\mathcal{D}_2$. Denote by $\mathcal{D}_{12}$ the database computed by A2M, by integrating $\mathcal{D}_1$ into $\mathcal{D}_2$. We evaluated the quality of $\mathcal{D}_{12}$ by the recall@ k of top- k vector search with query vector $\text{emb}_2(q)$, *i.e.*, the vector of q embedded by emb_2 . Here recall@ k is the percentage of (q, ans) pairs with ans documents identified by the top- k vector search with $\text{emb}_2(q)$ over $\mathcal{D}_{12}$.

The results of for various emb_1 and emb_2 models are shown in Table 1 (without the part shadowed in gray; see more in Section 7). Here Union is a baseline that directly takes the union of $\mathcal{D}_1$ and $\mathcal{D}_2$: *i.e.*, it uses identify function as f_{iso} in the A2M framework.

The results show that text retrieval quality over vectors integrated by A2M is similar or even worse (*e.g.*, on SciFact benchmark [80]) than that of Union. Indeed, Union guarantees that all ans documents that are distributed only in $\mathcal{O}_2$ are recalled via $\text{emb}_2(q)$ over the integrated database, since all vectors of $\mathcal{D}_1$ would be disregarded during retrieval as they are far away from vectors of $\mathcal{D}_2$. On the other hand, A2M transforms and places vectors of $\mathcal{D}_1$ in close proximity to vectors in $\mathcal{D}_2$, which can affect the recall performance over integrated database if they are not perfectly placed.

Answer to Q2. We observe that integrating $\mathcal{D}_1$ into $\mathcal{D}_2$ based on the Hypothesis A (via A2M) does not help improve the recall of vector search over $\mathcal{D}_2$ even though integration expands similarity search across datasets. As we will see in Section 7, the results are similar even if we replace the simple isometry f_{iso} with more sophisticated or even non-linear learned embedding alignment functions. This is consistent with the evaluation result of (Q1) that suggests different embedding vectors of the same objects $\mathcal{O}_\cap$ are not really isometric, rendering $f_{\text{iso}}(\mathcal{D}_1)$ inaccurate for $\mathcal{D}_2$.

Table 1. Recall@100 over integrated vector databases

emb ₁ vs emb ₂	method	Dataset			
		SciFact	NFCorpus	ArguAna	SciDocs
Fast Text vs NV-embed V2	Union	26.33	13.62	30.34	10.80
	A2M	18.33	8.05	25.91	5.70
	LA2M	76.67	34.06	64.60	39.30
NV-embed vs Openai Ada	Union	32.00	14.24	28.19	16.70
	A2M	32.00	14.24	28.19	16.70
	LA2M	79.33	35.29	85.22	41.10
gte-Qwen2 vs NV-embed V2	Union	31.67	18.27	28.48	19.60
	A2M	32.33	15.48	28.48	18.40
	LA2M	85.33	31.27	86.65	44.60
Mistral vs Openai Ada	Union	32.00	15.17	28.19	17.60
	A2M	27.00	10.84	29.91	12.60
	LA2M	76.67	34.06	79.23	42.00

4 Hypothesis Revisited for Vector Database Integration

From Section 3, we observe that Hypothesis A does not necessarily hold for vector databases. As a result, one cannot simply reduce the integration of vector databases to an application of existing embedding alignment techniques via A2M as this would yield an integrated database that cannot accurately implement top- k similarity search.

To rectify this, we first re-examine the hypothesis, showing that all we need to make Hypothesis A work for vector databases is a simple condition on vector distances of concern (Section 4.1). Based on it we then develop a new integration framework (Section 4.2).

4.1 Hypothesis: From Global Isometry to Local Isometries

We have seen from Section 3.2 that the integration of vector databases $\mathcal{D}_1$ and $\mathcal{D}_2$ for data items of $\mathcal{O}_1$ and $\mathcal{O}_2$, respectively, cannot serve as a vector database that encodes the universe $\mathcal{O}_1 \cup \mathcal{O}_2$, as shown by the performance of A2M in Table 1. This is mostly due to that different embedding models produce non-isometric vectors for common data items in $\mathcal{O}_1 \cap \mathcal{O}_2$ as shown in Fig. 3a of Section 3.1.

However, we are not completely fruitless. In fact, the experimental findings, especially those in Fig. 3b, seem to suggest that different embedding models behave quite consistently for *short distances*. Such short distances are also the key to implement accurate top- k similarity search for reasonable k .

As shown in Fig. 3b, over SciFact dataset when k is small, e.g., less than 5, top- k document retrieval results with NV-Embed-V2 embeddings and those with OpenAI-Ada are almost identical, with Jaccard Index consistently approaching 1. When k increases, this consistency between NV-Embed-V2 and OpenAI-Ada decreases significantly. This seems to suggest that NV-Embed-V2 and OpenAI-Ada encode closely related documents via consistent short vector distances, despite that they are entirely different embedding models. We capture this observation via a new hypothesis below.

Hypothesis B [The Local Isometry Hypothesis]: *The global isometry hypothesis (Hypothesis A) holds within local regions of vector databases comprised of vectors with short distances. □*

To validate if this holds, we first visually examine local structures of vector databases embedded by FastText [8] and NV-Embed-V2 [50] for the SciFact dataset. The zoomed-in T-SNE visualization of a query and its 10 nearest neighbors is shown in Fig. 4, where the left side depicts the distribution of results identified by top-10 vector search over FastText embedding vectors, and the right side shows the results of NV-Embed-V2 vectors. It is not difficult to see that the distribution of the top- k vector search results of the two embedding models are strikingly similarly: they produce exactly

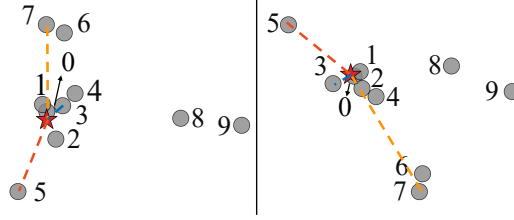


Fig. 4. A zoomed-in T-SNE view of a query (★) and its 10 nearest neighbors in SciFact (left shows the results over FastText [91] embeddings and right depicts NV-Embed-V2 [50] results.)

the same list of top-10 most similar SciFact documents, and these answers form an approximately isometric shape in the vector space. This seems to suggest that Hypothesis A(1) indeed holds in this local region of SciFact vectors embedded by FastText and NV-Embed-V2

To further evaluate if Hypothesis A(2) holds in local regions as Hypothesis B states above, we note that Hypothesis B naturally suggests a new approach to integrating vector databases produced by different embedding models, which we refer to as *localized* A2M (denoted by LA2M and illustrated in Fig. 2c): (1) we first decompose vector databases into clusters of vectors that have short inter-distances; (2) we then apply A2M of Section 3.2 independently to each cluster.

We then test the accuracy of integrated vector databases computed by LA2M, by using exactly the same setting as we evaluate A2M in Section 3.2. The results, as highlighted in gray in Table 1, show that LA2M significantly boosts the accuracy of A2M. For instance, its recall@100 of document retrieval results over integrated vector databases is consistently more than double of that of A2M and Union (see more results in Section 7). This further confirms that Hypothesis A(2) indeed holds in local regions of vector databases. These findings indicate that, for real-life vector databases, Hypothesis B is significantly more valid than Hypothesis A.

4.2 A Framework for Vector Database Integration

We next describe framework LA2M in detail, showing how we make “localized” use of embedding alignment to best exploit Hypothesis B for better integrated databases. We also point out key technical challenges in using LA2M, which we will tackle in subsequent sections.

Framework. LA2M takes as input two vector databases $\mathcal{D}_1$ and $\mathcal{D}_2$ and produces an integration mapping function $\mathcal{T}$ such that $\mathcal{T}(\mathcal{D}_1 \cup \mathcal{D}_2)$ serves as an integrated vector database for data items encoded by $\mathcal{D}_1$ and $\mathcal{D}_2$ taken together. Here $\mathcal{D}_1$ and $\mathcal{D}_2$ are possibly generated by two different embedding models emb_1 and emb_2 , respectively, for data item collections $\mathcal{O}_1$ and $\mathcal{O}_2$; however, LA2M assumes no access or knowledge to either emb_i or $\mathcal{O}_i (i \in \{1, 2\})$ when computing the integration function $\mathcal{T}$.

It does so by exploiting *reference* vectors between $\mathcal{D}_1$ and $\mathcal{D}_2$: pairs of vectors that encode the same objects, *i.e.*, $\mathcal{X}$ and $\mathcal{Y}$ in A2M of Section 3.2. Reference vectors are abundant in scenarios where we need to integrate vector databases, as databases must be relevant so that integration is beneficial. Typical sources of references include known correspondence in embedding backfilling, domain knowledge, or data linkage results. In case they are not known a priori, LA2M allows to automatically deduce such references (Section 6).

More specifically, LA2M integrates $\mathcal{D}_1$ into $\mathcal{D}_2$ in three steps:

Step (1): Grouping. It first groups the reference vectors $\mathcal{X}$ in $\mathcal{D}_1$ into clusters $X_1, \dots, X_n$, each with vectors that have short inter-distances. Denote by Y_i the set of matching vectors of X_i in $\mathcal{Y}$ of $\mathcal{D}_2$ ($i \in [1, n]$). Note that the clusters do not have to be disjoint.

Step (2): Alignment. Following Hypothesis B, LA2M deduces a collection of isometries to align the reference clusters independently. For each pair X_i and Y_i , it invokes A2M to compute an isometry f_{iso}^i that aligns X_i to Y_i as discussed in Section 3.2. One may also replace A2M with other embedding alignment variants; we use isometries here as they are easy to compute and offer us a provable bound on the quality of integrated vector database as we will see in Section 5.

Step (3): Merging. LA2M decides the integration mapping $\mathcal{T}$ for $\mathcal{D}_1$ with local isometries computed in step (2). Specifically, for any vector $\mathbf{x}$ in $\mathcal{D}_1$, $\mathcal{T}(\mathbf{x})$ maps $\mathbf{x}$ according to the position of $\mathbf{x}$:

- (a) $\mathbf{x}$ is a reference vector. When $\mathbf{x}$ is a reference vector in a cluster X_i grouped in step (1), $\mathcal{T}$ maps $\mathbf{x}$ to the vector $\mathbf{y}$ in Y_i of $\mathcal{D}_2$ that encodes the same data item as $\mathbf{x}$ does in X_i of $\mathcal{D}_1$.
- (b) $\mathbf{x}$ is a non-reference vector. Otherwise, $\mathbf{x}$ is a non-reference vector in $\mathcal{D}_1$ that needs to be mapped to the vector space of $\mathcal{D}_2$. $\mathcal{T}$ does so by picking X_{j^*} with centroid $\mathbf{c}_{j^*}$ that is closest to $\mathbf{x}$ among all reference clusters, i.e., $j^* = \arg \min_{i \in [1, n]} \|\mathbf{c}_i - \mathbf{x}\|$, where $\mathbf{c}_{j^*} = \sum_{\mathbf{x} \in X_{j^*}} \mathbf{x} / |X_{j^*}|$. It then uses $f_{iso}^{j^*}$ to map $\mathbf{x}$ to $\mathcal{D}_2$, i.e., $\mathcal{T}(\mathbf{x}) = f_{iso}^{j^*}(\mathbf{x})$.

Finally, LA2M returns $\mathcal{T}(\mathcal{D}_1) \cup (\mathcal{D}_2 \setminus \mathcal{Y})$ as the integrated database. With access to $\mathcal{D}_1$ and $\mathcal{D}_2$ only, LA2M expands the scope of top- k similarity search from over only data items encoded by $\mathcal{D}_2$ to over the universe of objects encoded by $\mathcal{D}_1$ and $\mathcal{D}_2$ taken together.

Benefits & implications. As shown in Table 1, LA2M enables us to integrate vector databases across embedding models, realizing cross-dataset similarity search. It also has diverse implications.

(1) *Model & object oblivious.* LA2M merges vector databases without integrating the data items encoded by the vectors, nor requiring access to the embedding models. Such obliviousness to the actual encoded datasets and embedding models makes LA2M a perfect fit for cases where we cannot freely share objects but still want to search across datasets, e.g., when sharing data objects can be expensive such as in edge devices or restricted due to privacy constraints.

(2) *Vector index invariant.* LA2M not only transforms and maps vectors of $\mathcal{D}_1$ to $\mathcal{D}_2$, it can also directly transforms vector indices built atop $\mathcal{D}_1$, e.g., IVF and HNSW, to the transformed vectors in $\mathcal{D}_2$ space. This is because LA2M uses isometries to align and merge vectors. In addition to preserving similarity search over $\mathcal{D}_1$, isometries can also transform IVF and HNSW over $\mathcal{D}_1$ as they are based on vector distances which are preserved during the merging step.

Key factors & challenges. The quality of the integrated database $\mathcal{T}(\mathcal{D}_1) \cup \mathcal{D}_2$ heavily relies on the following two factors of LA2M:

- *Availability of references* $\mathcal{X} \subseteq \mathcal{D}_1$ and $\mathcal{Y} \subseteq \mathcal{D}_2$.
- *Scope of isometries:* grouping $\mathcal{X}$ into $X_1, \dots, X_n$.

LA2M decides the isometries by analyzing reference vectors $\mathcal{X}$ and $\mathcal{Y}$ across $\mathcal{D}_1$ and $\mathcal{D}_2$. Such references are available from domain knowledge on the datasets, or can be deduced by employing entity linking methods [10, 71]. The rationale is that when we want to integrate $\mathcal{D}_1$ and $\mathcal{D}_2$, the data items encoded by them must be correlated; such correlation offers potential for the existence of linkage across the datasets, which, mapped to the vectors, gives us the reference vectors $\mathcal{X}$ and $\mathcal{Y}$. In the worst case where $\mathcal{X}$ and $\mathcal{Y}$ are not available, we develop an interface for LA2M to actively generate references with minimum access to the datasets (Section 6.2).

Even more challenging is to decide how we shall group $\mathcal{X}$ into local clusters $X_1, \dots, X_n$, to maximize the quality of integrated vector database $\mathcal{T}(\mathcal{D}_1) \cup \mathcal{D}_2$. As shown in Fig. 5, the scope of X_i evidently affects the errors that LA2M produces in step (2) and step (3), which we refer to as the *alignment* error α and *integration* error β , respectively. To deal with this, we first theoretically

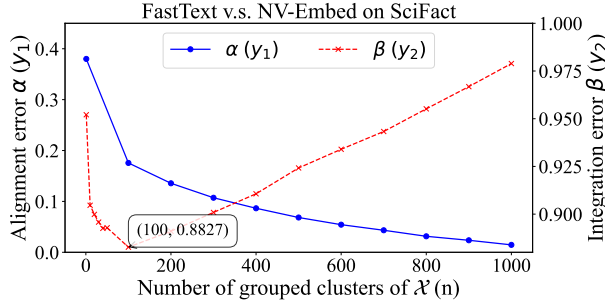


Fig. 5. #-clusters of $\mathcal{X}$ vs alignment error α and integration error β . (alignment error α : distance between $f_{\text{iso}}^j(\mathbf{x})$ and $\mathbf{y}$ when $\mathbf{x} \in X_j$ corresponds to $\mathbf{y} \in Y_j$ (step (2) error); β : distance between $\mathcal{T}(\mathbf{x})$ and the vector would have been embedded by emb_2 (step (3) error). $\mathcal{X}$ is grouped with k-meaning clustering.)

analyze and bound the errors on mappings $\mathcal{T}(\mathbf{x})$ of vectors $\mathbf{x} \in \mathcal{D}_1$ in Section 5, based on which we then discuss how LA2M breaks down $\mathcal{X}$ in Section 6.1.

5 Bounding the Quality of Integrated Vector Databases

To decide key design factors of LA2M, we study the quality of integrated vector databases.

Quantifying integration quality. We first consider metrics to quantify the quality of vector databases integrated by LA2M. While recall@k as reported in Table 1 (and also Section 7) is a standard choice for similarity search, it is sensitive to the curated query-answer samples built in the benchmarks; moreover, its empirical nature offers little insight in the source and properties of the errors.

To this end, below we use vector distances directly to quantify the error of every transformed vector of $\mathcal{D}_1$ in the vector space of $\mathcal{D}_2$. Recall from Section 4.2 that LA2M transforms vectors of $\mathcal{D}_1$ embedded by model emb_1 and merges them into database $\mathcal{D}_2$ embedded by model emb_2 via function $\mathcal{T}$, in the form of a collection of isometries. Each isometry f_{iso}^i is determined by a local cluster X_i of reference vectors that corresponds to its counterpart Y_i in $\mathcal{D}_2$.

Integration error (β). Consider an arbitrary non-reference vector $\mathbf{x}$ of $\mathcal{D}_1$. We call $\mathcal{T}(\mathbf{x})$ its image w.r.t. $\mathcal{T}$ in the space of $\mathcal{D}_2$. Let $\mathbf{o}_x$ be the data item that it encodes in $\mathcal{D}_1$. Then the *integration error* $\beta(\mathbf{x})$ of $\mathcal{T}$ is the distance between $\mathcal{T}(\mathbf{x})$ and $\text{emb}_2(\mathbf{o}_x)$, i.e., the imagined “ideal” vector that emb_2 would have generated in $\mathcal{D}_2$ if it were asked to embed $\mathbf{o}_x$. It should be noted that $\text{emb}_2(\mathbf{o}_x)$ is a hypothetical vector that is not part of $\mathcal{D}_2$, but it resides in the same ambient space.

Note that the integration error β is defined for all vectors of $\mathcal{D}_1$, and reflects the divergence between the integrated vectors and their “ideal” position in the vector space of $\mathcal{D}_2$. As shown in Fig. 5, it is sensitive to how we group references $\mathcal{X}$ of $\mathcal{D}_1$ in step (1) of LA2M.

To reveal their connections, we further delve into the contributing factors of β . In particular, the integration error of $\mathbf{x}$, i.e., $\|\mathcal{T}(\mathbf{x}) - \text{emb}_2(\mathbf{o}_x)\|$, is the combined effect of two sources of errors:

- *Model disagreement*: the inherent difference between emb_1 and emb_2 , i.e., even if both models embed the same dataset, their produced vector databases would not align with each other.
- *Merging error*: the “generalization error” of isometry f_{iso}^i determined by references X_i for transforming and integrating vector $\mathbf{x}$ that are not in X_i .

Intuitively, model disagreement is inherent to embedding models and is hence constant w.r.t. any given $\mathcal{D}_1$ and $\mathcal{D}_2$. Computationally, this is reflected by that even the optimal isometry f_{iso}^i cannot exactly align all references in X_i of $\mathcal{D}_1$ to their images in Y_i of $\mathcal{D}_2$ by step (2) of LA2M. Hence, the optimal